

Sistemi di Calcolo (A.A. 2024-2025)

Corso di Laurea in Ingegneria Informatica e Automatica
Sapienza Università di Roma



Compito (14/07/2025) – Durata 1h 30'

Inserire nome, cognome e matricola nel file `studente.txt`.

ISTRUZIONI PER STUDENTI DSA: svolgere a scelta due parti su tre.

Parte 1 (programmazione IA32)

La funzione `longest_string` calcola la lunghezza della stringa più lunga contenuta in un array di caratteri. Nella directory E1, si traduca in assembly IA32 la seguente funzione C scrivendo un modulo `e1A.s`:

```
unsigned int longest_string(const char* data, int len) {
    int index = 0;
    int max = -1;

    while (index < len){
        int x = call_strlen(data + index);
        if (x > 0 && x > max)
            max = x;
        index = index + x + 1;
    }
    return max;
}
```

L'unico criterio di valutazione è la correttezza. Generare un file eseguibile `e1A` con `gcc -m32 -g` (per evitare possibili warning è possibile aggiungere lo switch `-no-pie`). Per i test, compilare il programma insieme al programma di prova `e1A_main.c`.

Non modificare in alcun modo `e1A_main.c`. Prima di tradurre il programma in IA32 si suggerisce di scrivere nel file `e1A_eq.c` una versione C equivalente più vicina all'assembly.

Parte 2 (programmazione di sistema POSIX)

Si scriva una funzione che processi parallelamente un array di stringhe suddividendole in coppie. Nello specifico, si scriva nel file `E2/e2A.c` una funzione con il seguente prototipo (definito in `E2/e2A.h`):

```
int countCouples(const char** s, int k, int l);
```

che, dato un array `s` di stringhe che termina sempre con `NULL`, calcoli tutte le coppie possibili delle prime `k` stringhe di `s` e ritorni il numero di quelle coppie le cui stringhe condividono un prefisso di lunghezza almeno uguale a `l`. La verifica dei prefissi deve essere svolta in parallelo sulle coppie generate, sfruttando i **thread**.

Dato un array di `n` elementi, è possibile formare, $n^2 - n$ coppie di elementi (escludiamo le coppie con elementi identici). Ad esempio, se l'array di stringhe fosse:

```
s = { "hello", "world", "hello world", NULL }
```

abbiamo 6 possibili coppie di stringhe:

```

c1 = { "hello", "world" } c2 = { "hello", "hello world" }
c3 = { "world", "hello" } c4 = { "world", "hello world" }
c5 = { "hello world", "hello" } c6 = { "hello world", "world" }

```

Supponiamo ora di invocare la funzione in questo modo: `countCouples(s, 3, 3)`. In questo caso il valore restituito da `countCouples` sarà 2 perché due sole coppie tra le possibili 6 contengono stringhe con lo stesso prefisso di lunghezza almeno 3, ovvero c2 e c5:

```

c2 = { "hello", "hello world" } c5 = { "hello world", "hello" }

```

Una diversa invocazione, `countCouples(s, 3, 6)`, restituirebbe invece 0 in quanto nessuna coppia di stringhe condivide un prefisso di lunghezza 6.

Nel caso in cui k sia minore delle stringhe disponibili in s , come già anticipato, solo le prime k stringhe devono essere considerate per generare le coppie; se invece k è maggiore del numero delle stringhe oppure $k < 2$, la funzione deve segnalare l'errore restituendo -1. L'analisi delle coppie dovrà avvenire **in modo parallelo**, ossia in modo tale che la presenza di un prefisso comune in ciascuna coppia di stringhe venga controllata da un **thread differente**.

Nota: Si ricorda di utilizzare il flag **-lpthread** per la compilazione quando si utilizzano i thread POSIX.

Suggerimento: per passare a un thread più di un argomento si può utilizzare una struttura dati e poi passarne al thread un puntatore alla stessa. Un'ipotetica struct, in questo caso specifico, potrebbe avere la seguente forma:

```

typedef struct {
    char* s1; // stringa 1
    char* s2; // stringa 2
    int l; // lunghezza prefisso
    int result; // risultato del calcolo
} ThreadData;

```

Per i test, compilare il programma insieme al programma di prova `e2A_main.c` fornito, che **non** deve essere modificato.

Parte 3 (quiz)

Si risponda ai seguenti quiz, inserendo le risposte (A, B, C, D o E per ogni domanda) nel file `e3A.txt`. Una sola risposta è quella giusta. Rispondere E equivale a non rispondere (0 punti).

Domanda 1 (cache)

Una cache a mapping diretto ha 8 linee e blocchi da 64 byte. Quale linea conterrà l'indirizzo `0x01F4`?

A	4	B	5
C	6	D	7

Motivare la risposta nel file `M1.txt`. **Risposte non motivate saranno considerate nulle.**

Domanda 2 (ottimizzazione)

Cosa fa il loop unrolling?

A	Aumenta la profondità dei cicli	B	Trasforma cicli in ricorsioni
C	Elimina cicli annidati	D	Replica k volte il corpo del ciclo per ridurre il numero di iterazioni

Motivare la risposta nel file M2.txt. **Risposte non motivate saranno considerate nulle.**

Domanda 3 (file system)

Una directory ha permessi `rwX--X--X`. Cosa può fare un utente che non è né proprietario né nel gruppo?

A	Leggere i nomi dei file	B	Creare solo file nella directory
C	Accedere alla directory e ai soli file di cui conosce i nomi	D	Creare file e sottodirectory nella directory

Motivare la risposta nel file M3.txt. **Risposte non motivate saranno considerate nulle.**

Domanda 4 (paginazione)

Uno spazio logico a 30 bit usa una tabella delle pagine con 2048 entry. Sapendo che ogni entry è larga 4 byte, quanto deve essere grande ogni pagina?

A	2 K	B	8 KB
C	16 KB	D	512 KB

Motivare la risposta nel file M4.txt. **Risposte non motivate saranno considerate nulle.**